Girls Day 2026

Getting started with MicroPython

Die Quellen

https://micropython.org/

https://thonny.org/

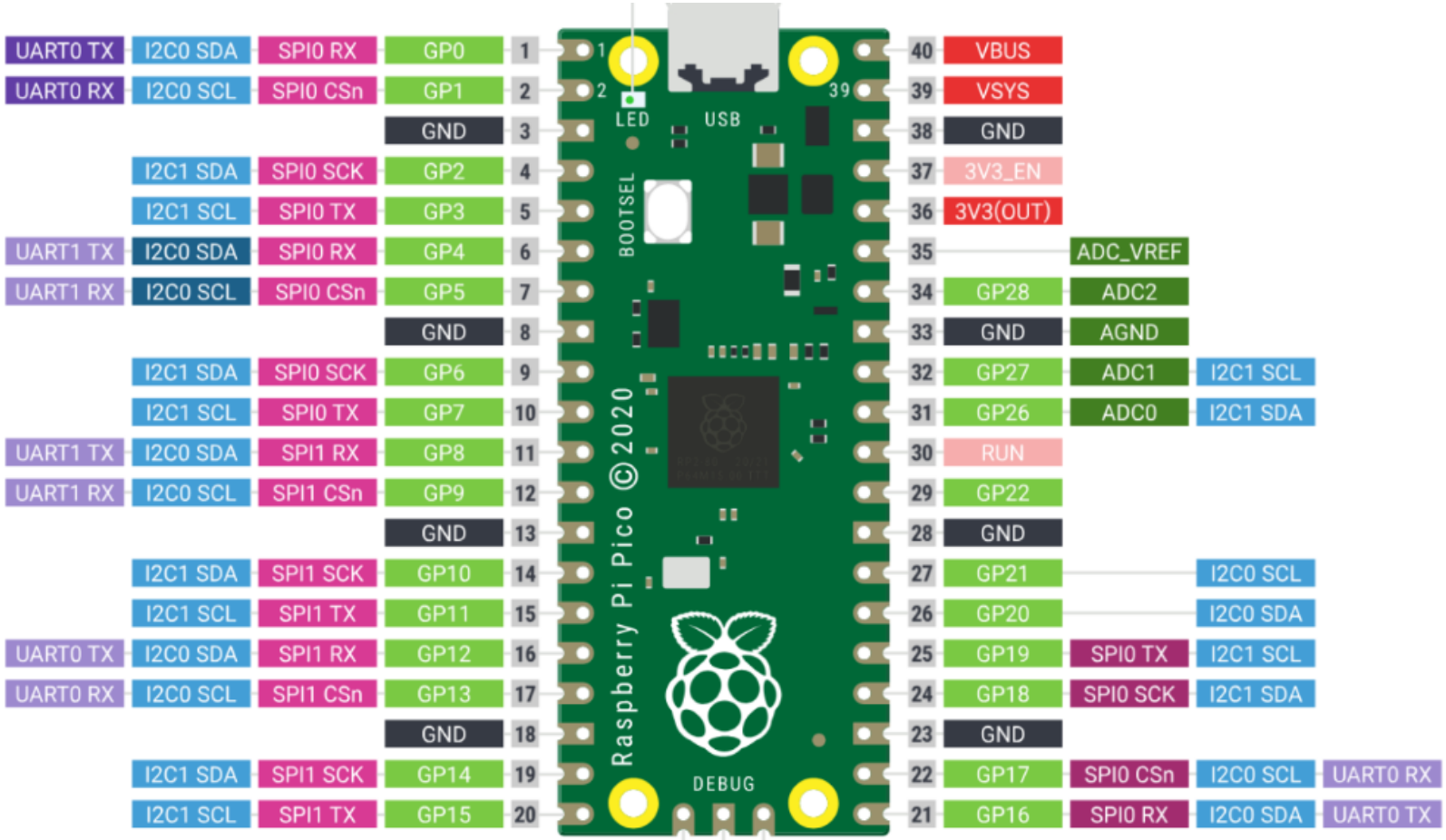
https://wokwi.com/

https://github.com/raspberrypipress/released-pdfs
=> get-started-with-micropython-raspberry-pi-pico.pdf

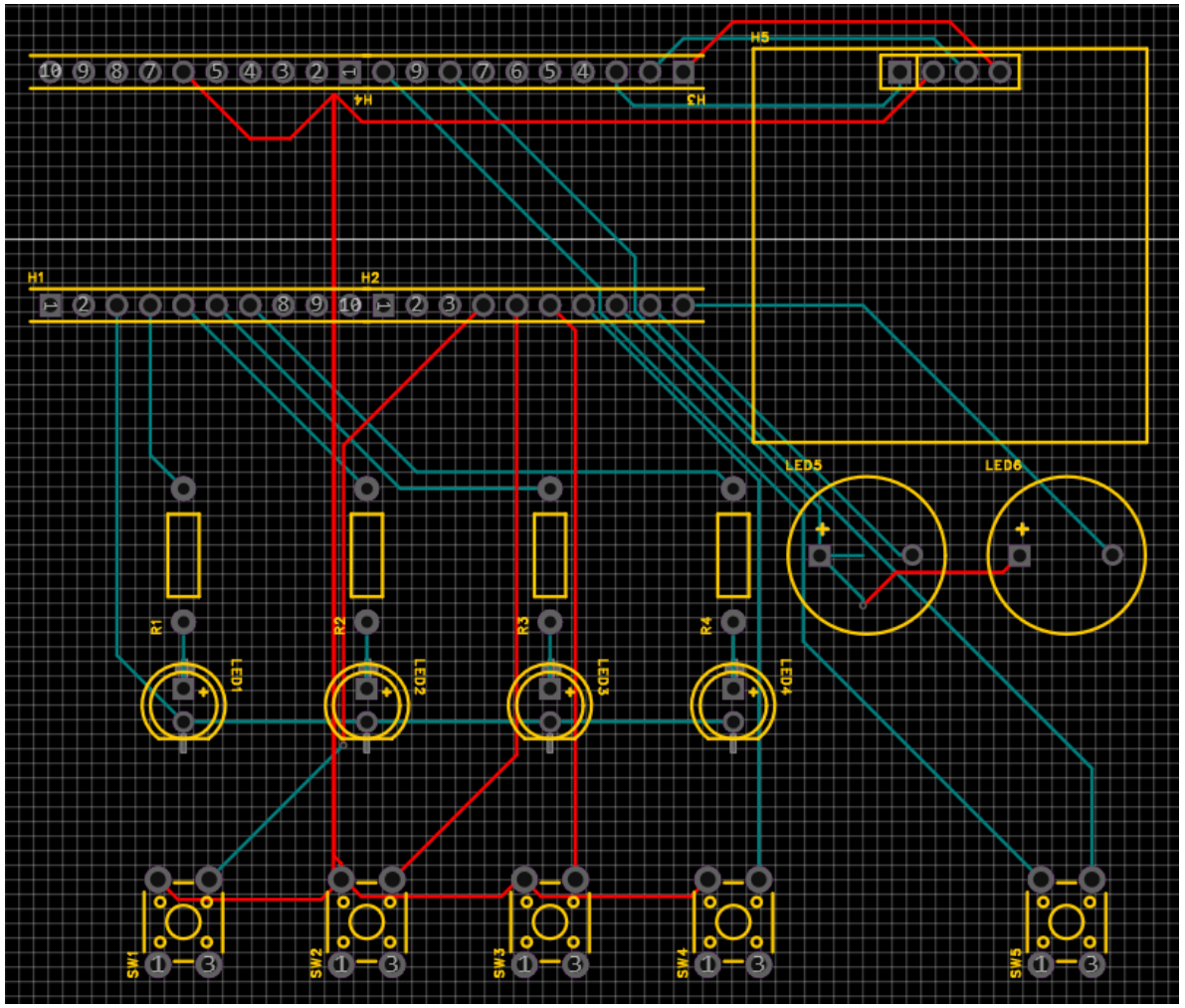
https://www.raspberrypi.com/
=> Documentation => Microcontrollers

https://nerdcave.xyz
=> Enter Pico Cave => Tutorials => Simon Says Game

Raspberry Pico Board

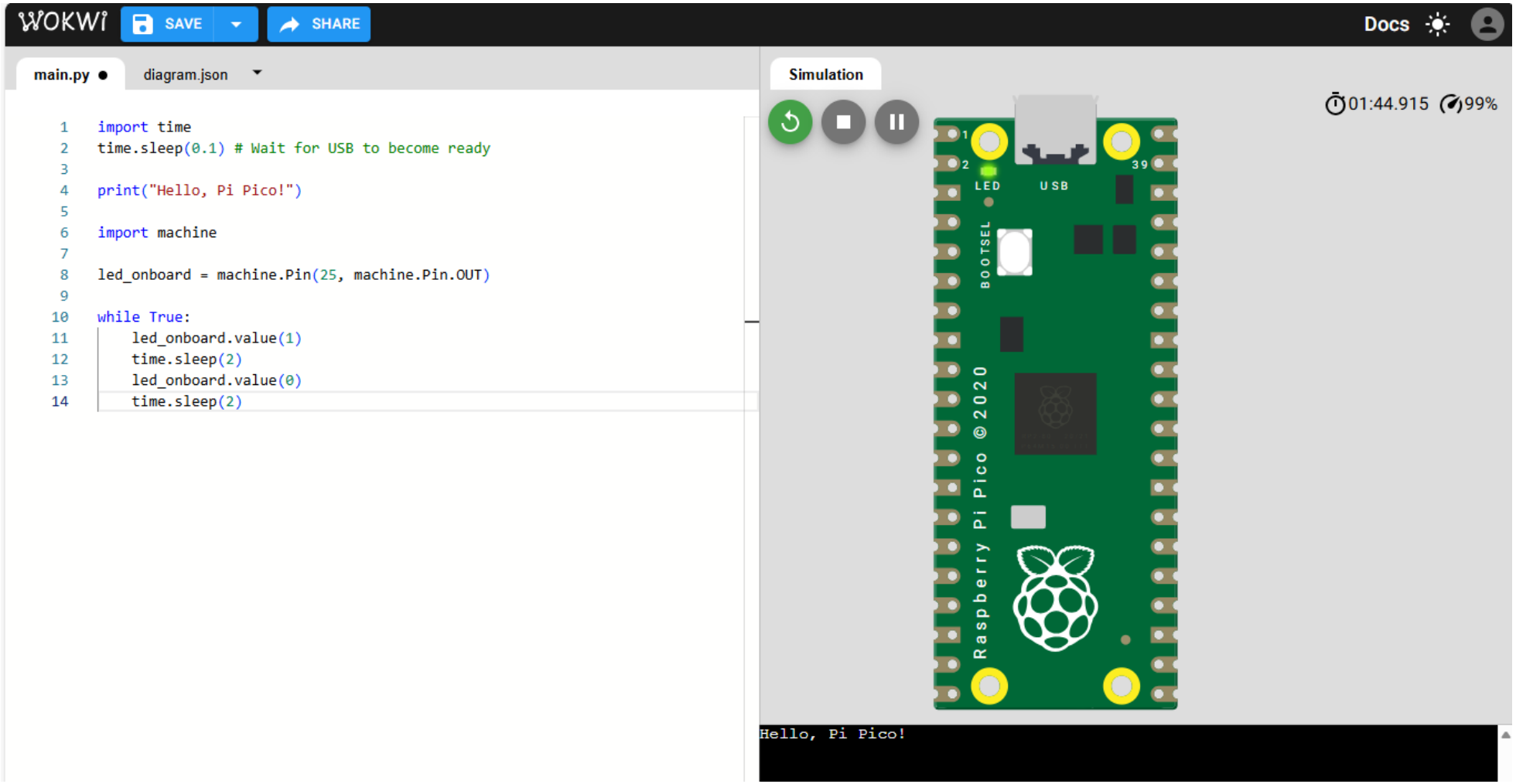


PCB Layout

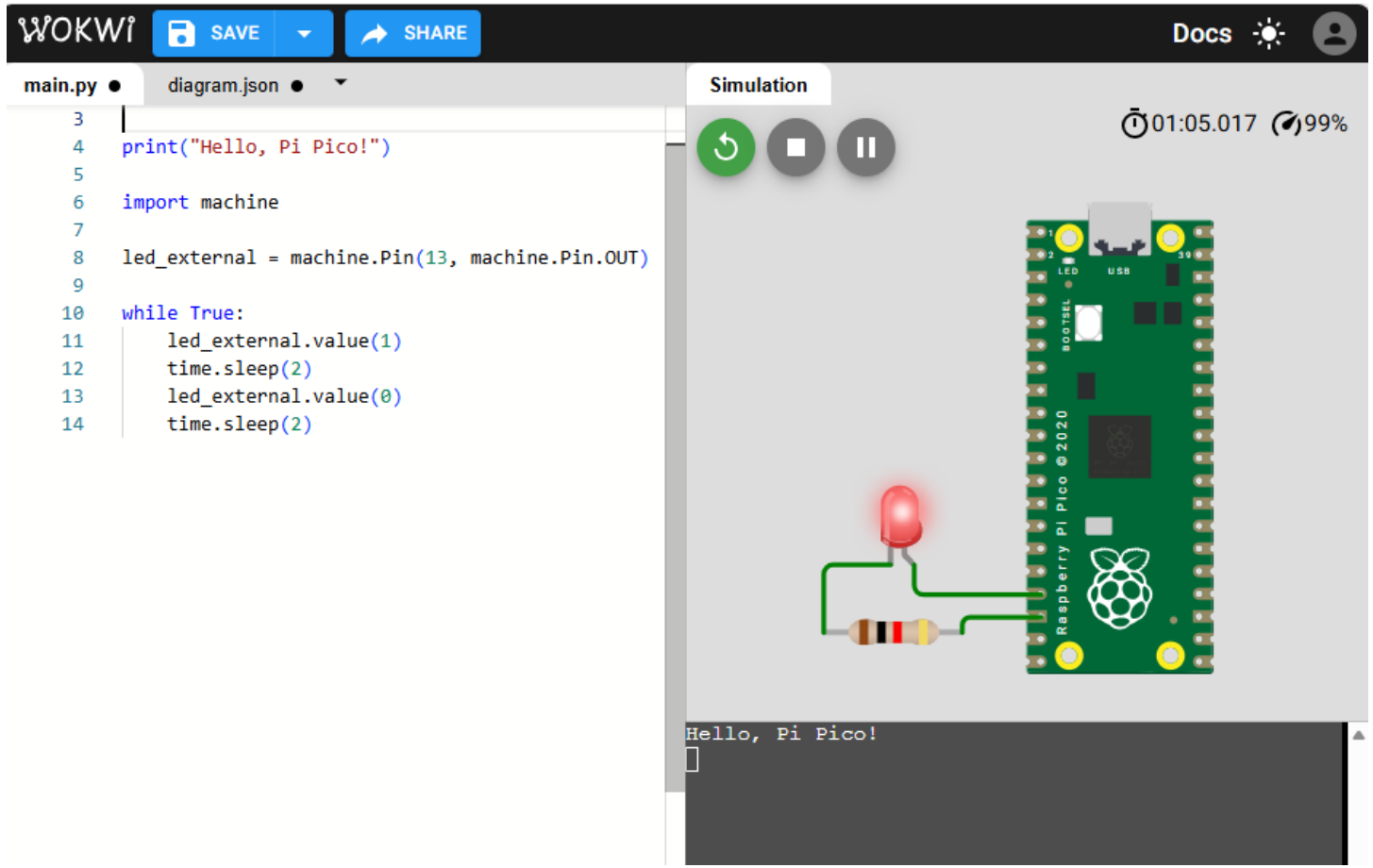

H5
H4
H3
H1
H2
LED5
LED6
R1
R2
R3
R4
LED1
LED2
LED3
LED4
SW1
SW2
SW3
SW4
SW5

Wokwi

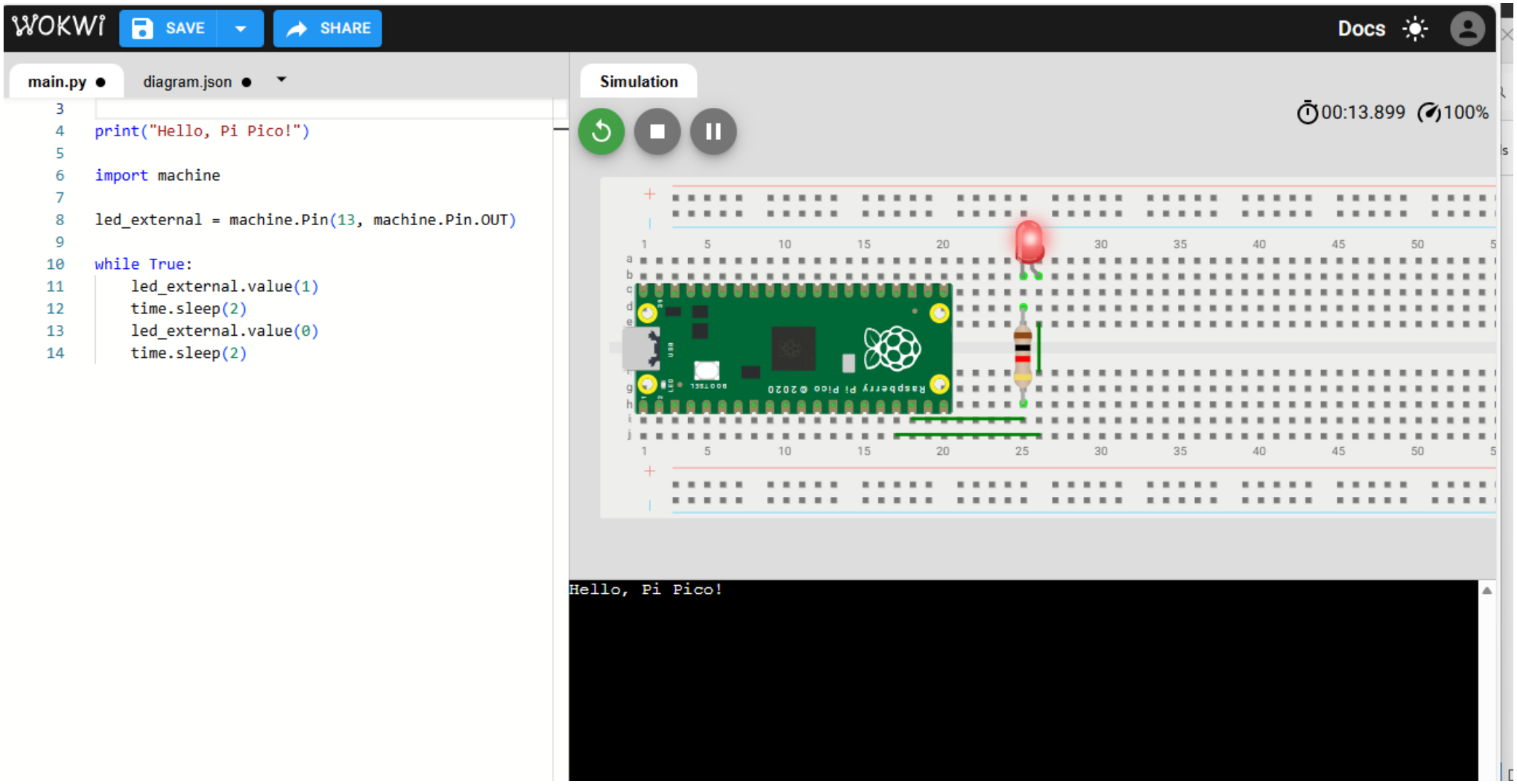
Hello World in Wokwi



Hello World in Wokwi



Hello World in Wokwi



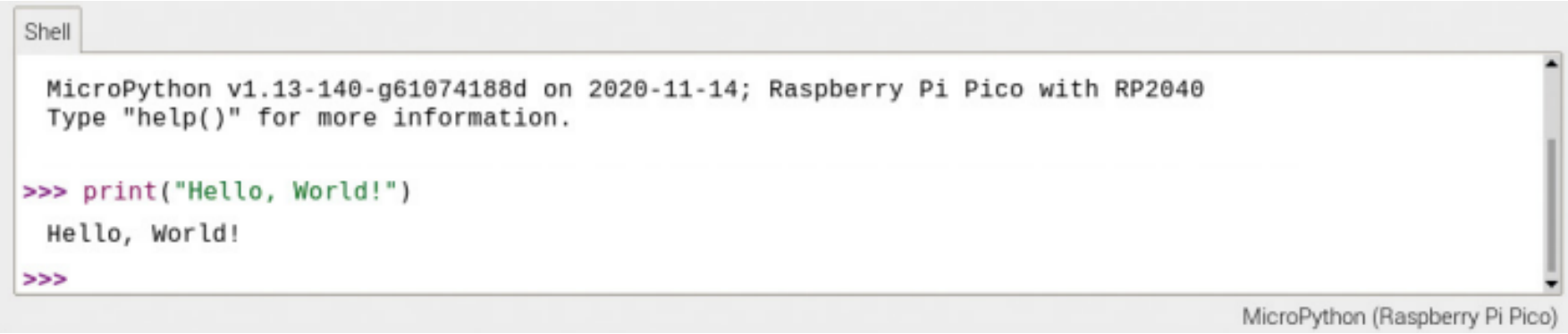
MicroPython

MicroPython

- Installation der Micropython Firmware
- Rufe www.micropython.org auf und lade aktuelle Firmware herunter. Diese liegt aus uf2-Datei vor.
- Halte den BOOTSEL Schalter gedrückt, schließe den Pico via USB Kabel an den Rechner und lasse den BOOTSEL Schalter los
- Der Pico erscheint als USB Stick
- Kopiere die uf2-Datei auf den USB Stick
- Der Pico startet neu und ist nicht mehr als USB Stick sondern als serielle Schnittstelle sichtbar

MicroPython

- Lade die zip-Datei des portablen 64-bit Thonny auf dem Laptop
- Entpacke die zip-Datei z.B. auf dem Desktop
- Starte Thonny
- Verbinde Thonny mit dem Pico
- Schreibe den folgenden Befehl im Interpreter



MicroPython

- Loop-Statement

```
print("Loop starting!")
for i in range(10):
    print("Loop number", i)
print("Loop finished!")
```

```
Loop starting!
Loop number 0
Loop number 1
Loop number 2
Loop number 3
Loop number 4
Loop number 5
Loop number 6
Loop number 7
Loop number 8
Loop number 9
Loop finished!
```

MicroPython

- Conditional-Statement

[Hello World.py] [Indentation.py] [Name Test.py]

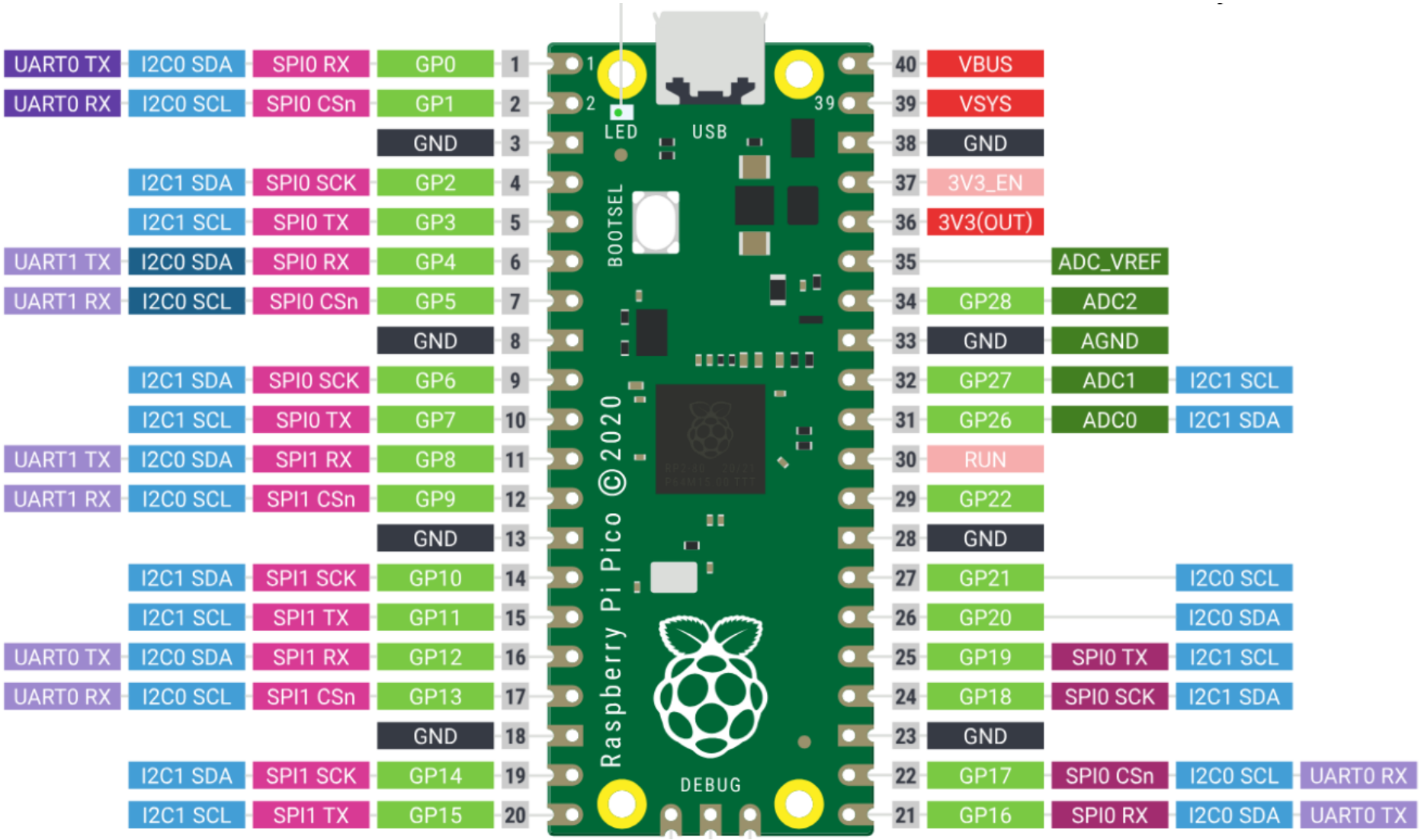
```
1 user_name = input ("What is your name? ")
2 while user_name != "Clark Kent":
3     print("You are not Superman - try again!")
4     user_name = input("What is your name? ")
5 print("You are Superman!")
```

```
>>> %Run -c $EDITOR_CONTENT

  What is your name? Gareth
  You are not Superman - try again!
  What is your name? Ben
  You are not Superman - try again!
  What is your name? Clark Kent
  You are Superman!
```

Hardware

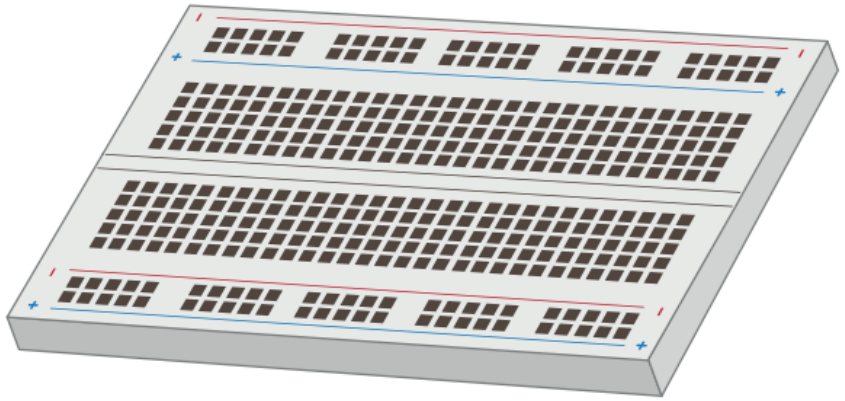
Hardware

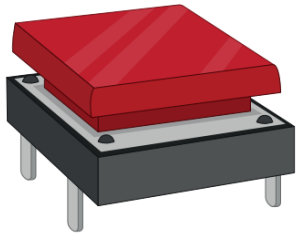
- Pinout

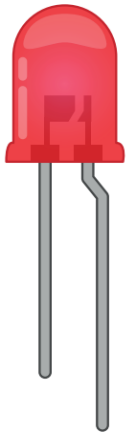


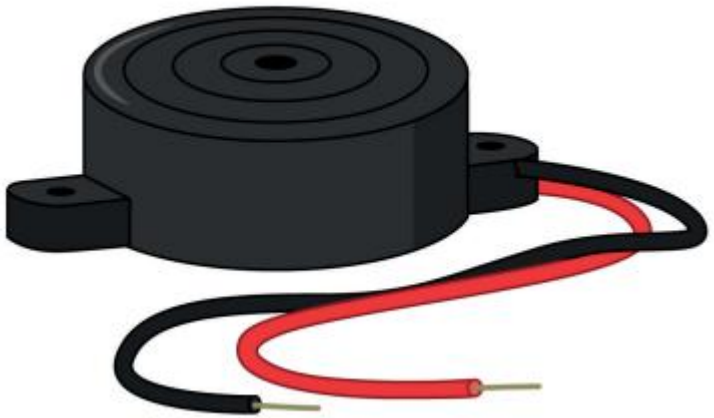
Hardware

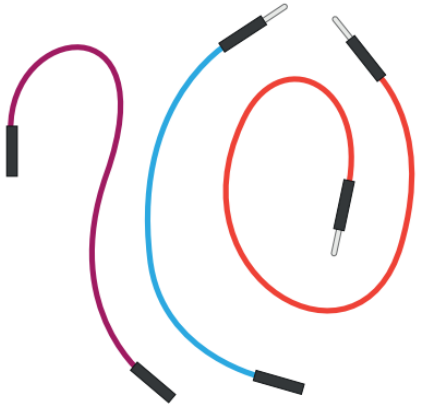
- Breadboard, Wire, LED, Resistor, Button, Buzzer











Hardware

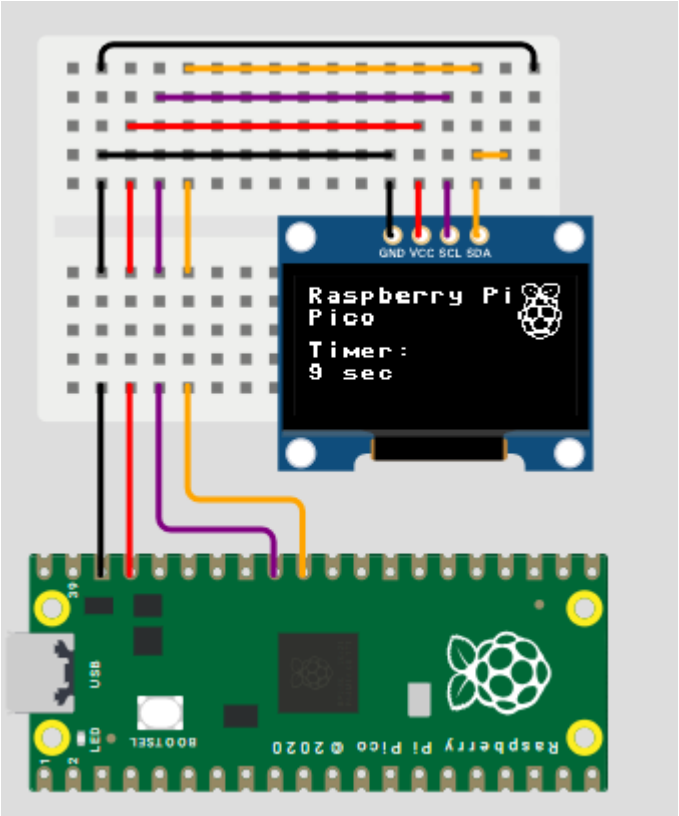
All the holes in each rail are connected
The five holes in each row are connected

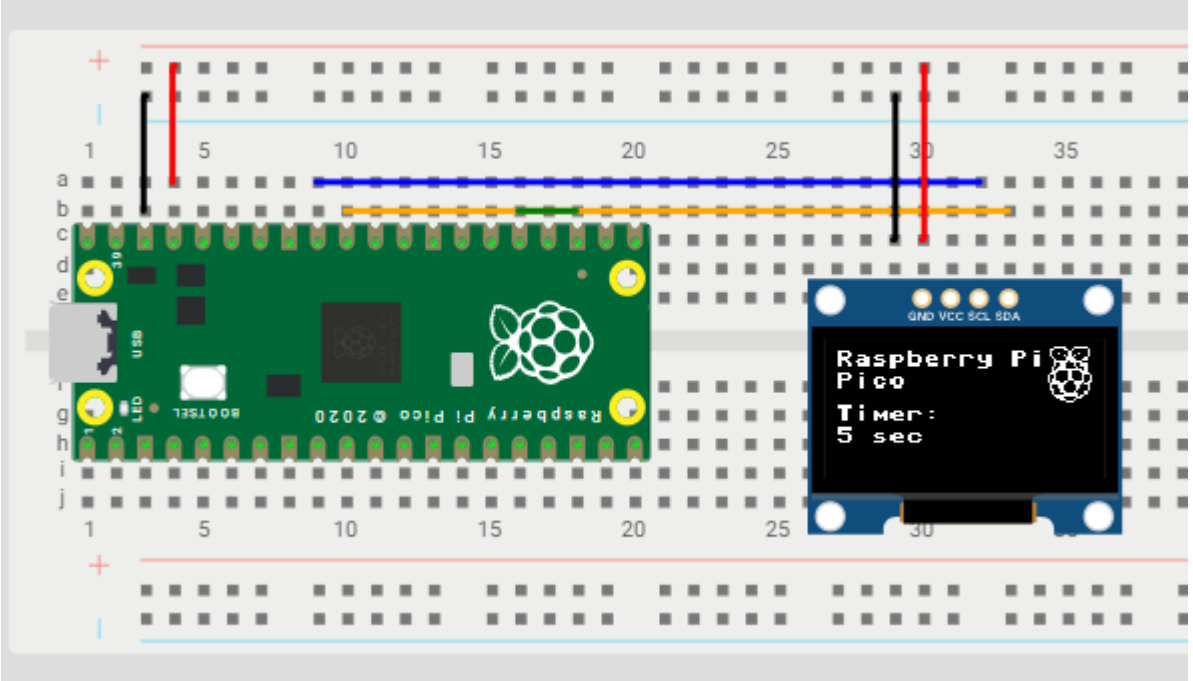
Challenge I

- Baue die folgende Wokwi Simulation auf dem Breadboard nach
    - Installiere die SSD1306 Library
    - Kopiere die Datei main.py auf den Pico
    - Führe einen Reset des Pico durch

Hardware

- OLED SSD1306 (https://wokwi.com/projects/359558101922696193)





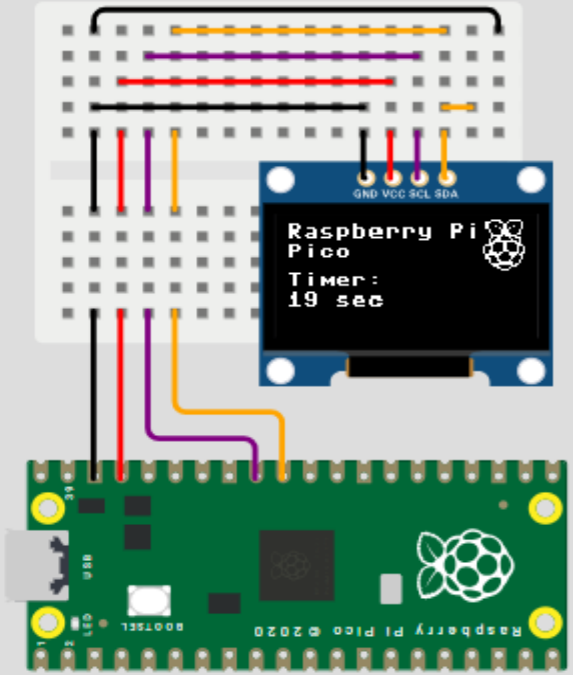
Challenge I

WOKWI SAVE SHARE SSD1306 with MicroPython by arcostasi

main.py diagram.json ssd1306.py PIO

```python
# MicroPython SSD1306 OLED driver, I2C and SPI interfaces
#
# library taken from repository at:
# https://github.com/micropython/micropython/blob/master/drivers/display/
#
from micropython import const
import framebuf
import time

# Register definitions
SET_CONTRAST = const(0x81)
SET_ENTIRE_ON = const(0xA4)
SET_NORM_INV = const(0xA6)
SET_DISP = const(0xAE)
SET_MEM_ADDR = const(0x20)
SET_COL_ADDR = const(0x21)
SET_PAGE_ADDR = const(0x22)
SET_DISP_START_LINE = const(0x40)
SET_SEG_REMAP = const(0xA0)
SET_MUX_RATIO = const(0xA8)
SET_COM_OUT_DIR = const(0xC0)
SET_DISP_OFFSET = const(0xD3)
```

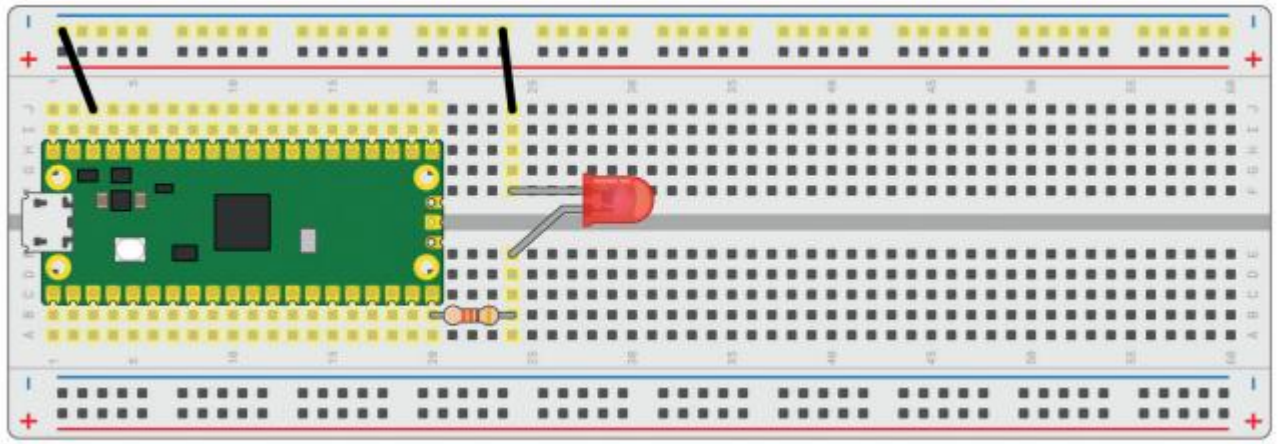
Simulation



Breadboard Schaltungen

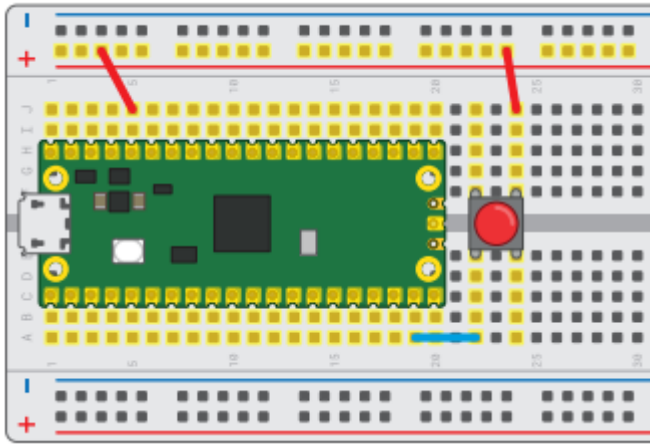
Breadboard Schaltungen

```
import machine
import utime

led_external = machine.Pin(15, machine.Pin.OUT)

while True:
    led_external.toggle()
    utime.sleep(5)
```



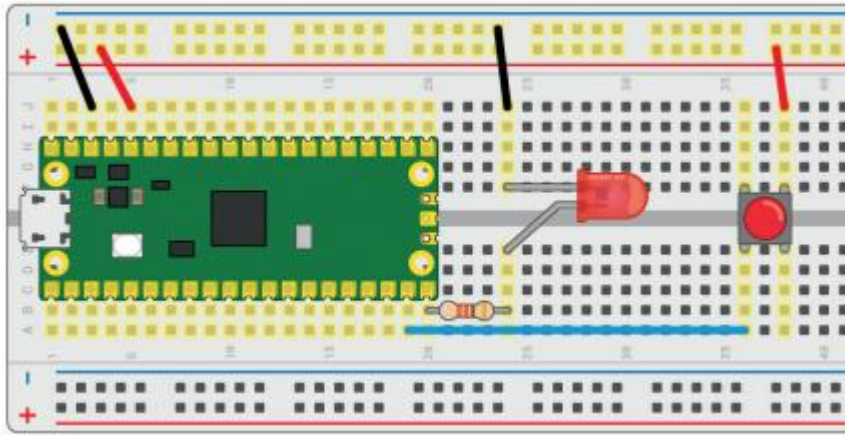
Breadboard Schaltungen

```python
import machine
import utime

button = machine.Pin(14, machine.Pin.IN, machine.Pin.PULL_DOWN)

while True:
    if button.value() == 1:
        print("You pressed the button!")
        utime.sleep(2)
```



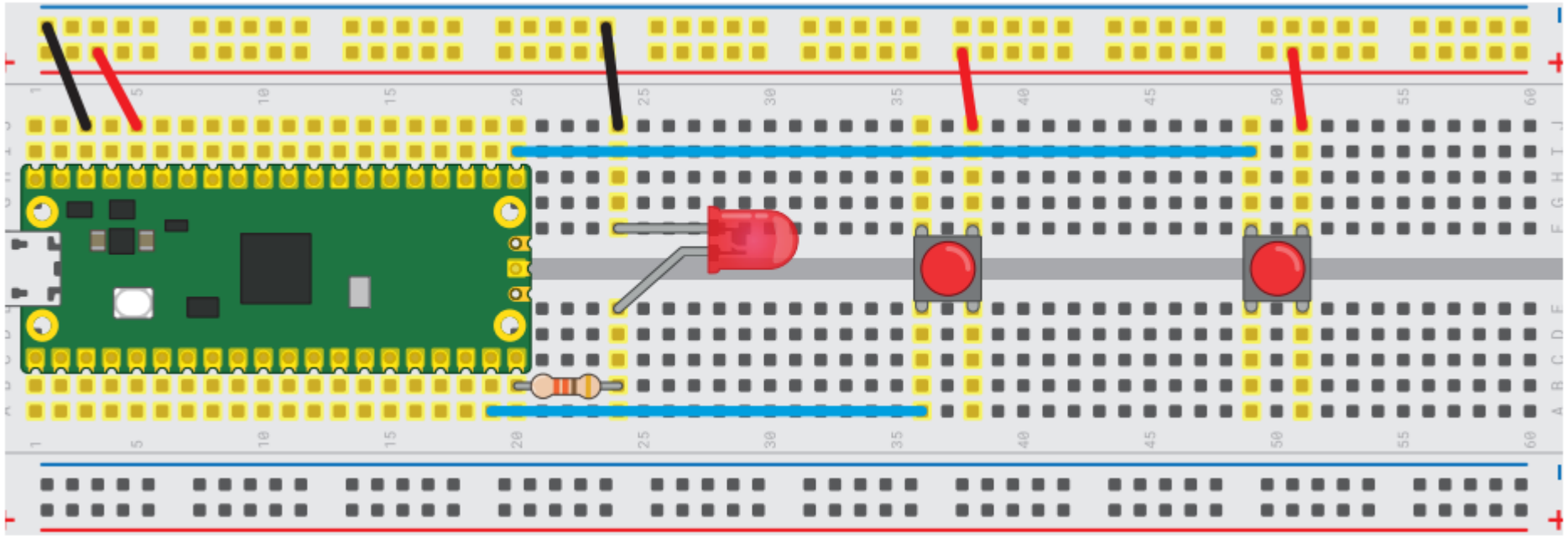
Breadboard Schaltungen

```
import machine
import utime

led_external = machine.Pin(15, machine.Pin.OUT)
button = machine.Pin(14, machine.Pin.IN, machine.Pin.PULL_DOWN)

 while True:
     if button.value() == 1:
         led_external.value(1)
         utime.sleep(2)
     led_external.value(0)
```



Reaction Game

Reaction Game



Reaction Game I

```
import machine
import utime
import urandom

led = machine.Pin(15, machine.Pin.OUT)
left_button = machine.Pin(14, machine.Pin.IN, machine.Pin.PULL_DOWN)
right_button = machine.Pin(16, machine.Pin.IN, machine.Pin.PULL_DOWN)

def button_handler(pin):
    left_button.irq(handler=None)
    right_button.irq(handler=None)
    timer_reaction = utime.ticks_diff(utime.ticks_ms(), timer_start)
    print("Your reaction time was " + str(timer_reaction) +
" milliseconds!")

led.value(1)
utime.sleep(urandom.uniform(5, 10))
led.value(0)
right_button.irq(trigger=machine.Pin.IRQ_RISING, handler=button_handler)
left_button.irq(trigger=machine.Pin.IRQ_RISING, handler=button_handler)
```

Reaction Game II

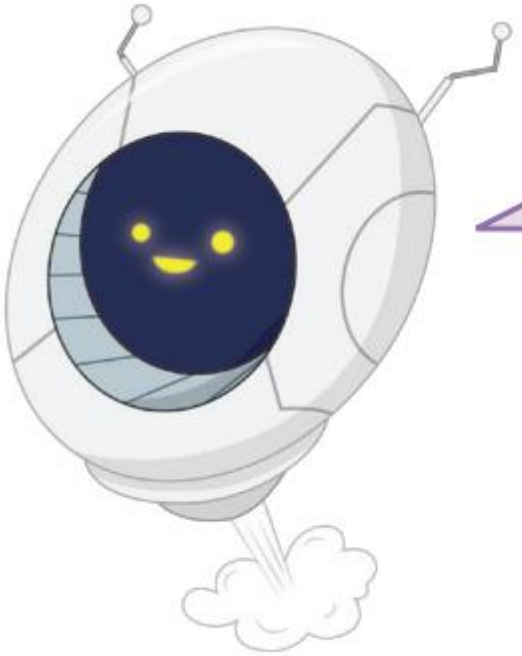
```
import machine
import utime
import urandom

led = machine.Pin(15, machine.Pin.OUT)
left_button = machine.Pin(14, machine.Pin.IN, machine.Pin.PULL_DOWN)
right_button = machine.Pin(16, machine.Pin.IN, machine.Pin.PULL_DOWN)

fastest_button = None

def button_handler(pin):
    left_button.irq(handler=None)
    right_button.irq(handler=None)
    global fastest_button
    fastest_button = pin
```

Reaction Game II

```
led.value(1)
utime.sleep(urandom.uniform(5, 10))
led.value(0)
left_button.irq(trigger=machine.Pin.IRQ_RISING, handler=button_handler)
right_button.irq(trigger=machine.Pin.IRQ_RISING, handler=button_handler)
while fastest_button is None:
    utime.sleep(1)
if fastest_button is left_button:
    print("Left Player wins!")
elif fastest_button is right_button:
    print("Right Player wins!")
```

Reaction Game II

- Can you add the timer back into your program?



CHALLENGE: TIMINGS

Can you modify the messages that print? Can you add a third button, so that three people can play at once? Is there an upper limit to how many buttons you could add? Can you add the timer back into your program, so it tells the winning player how quick their reaction time was?

Simon Says Hardware

Simon Says Hardware

```python
#https://nerdcave.xyz/docs/tutorials/simon-says-game/#main

import machine
import time
import random

from machine import Pin, I2C
from ssd1306 import SSD1306_I2C
import framebuf

# Define GPIO pins for LEDs and buttons
#LED_PINS = [12, 13, 14, 15]  # Blue, Red, Yellow, Green
#BUTTON_PINS = [11, 10, 9, 8]  # Blue, Red, Yellow, Green
#BUZZER_PIN = 6                # Buzzer pin
LED_PINS = [2, 3, 4, 5]  # Blue, Red, Yellow, Green
BUTTON_PINS = [10, 11, 12, 13]  # Blue, Red, Yellow, Green
BUZZER_PIN = 14                # Buzzer pin
ALARM_PIN = 15

# Initialize LEDs, buttons, and buzzer
leds = [machine.Pin(pin, machine.Pin.OUT) for pin in LED_PINS]
buttons = [machine.Pin(pin, machine.Pin.IN, machine.Pin.PULL_DOWN) for pin in BUTTON_PINS]
buzzer = machine.PWM(machine.Pin(BUZZER_PIN))

alarm = machine.Pin(ALARM_PIN, machine.Pin.OUT)

for i in range(6):
    alarm.toggle()
    time.sleep(0.2)

alarm.off()

# Define frequencies for each LED color (in Hz)
FREQUENCIES = [440, 494, 523, 587]  # A4, B4, C5, D5

WIDTH  = 128                                          # oled display width
HEIGHT = 64                                            # oled display height

i2c = I2C(0, scl=Pin(17), sda=Pin(16), freq=200000)       # Init I2C using pins GP8 & GP9 (defau
print("I2C Address      : "+hex(i2c.scan()[0]).upper()) # Display device address
print("I2C Configuration: "+str(i2c))                   # Display I2C config


oled = SSD1306_I2C(WIDTH, HEIGHT, i2c)                  # Init oled display

# Game variables
sequence = []
user_input = []
speed = 2 # Start speed

def display(line1, line2):
    oled.fill(0)
    oled.text(line1,5,5)
    oled.text(line2,5,15)
    oled.show()


def play_tone(frequency):
    buzzer.freq(frequency)  # Set frequency
    #buzzer.duty_u16(512)        # Set duty cycle (50% - adjust as needed)
```

Simon Says Hardware

```python
    buzzer.duty_u16(32768)
    time.sleep(0.2)         # Play tone for 0.5 seconds
    buzzer.duty_u16(0)          # Stop the tone

def light_led(index):
    leds[index].on()
    play_tone(FREQUENCIES[index])  # Play the corresponding tone
    time.sleep(0.05)  # LED on for 0.5 seconds
    leds[index].off()
    time.sleep(0.05)  # Short pause between LEDs

def flash_button(index):
    # Flash the button LED while pressed
    leds[index].on()
    play_tone(FREQUENCIES[index])  # Play the corresponding tone when button is pressed
    time.sleep(0.05)
    leds[index].off()

def get_user_input():
    global user_input
    user_input = []

    while len(user_input) < len(sequence):
        for i, button in enumerate(buttons):
            if button.value() == 1:  # Button is pressed
                flash_button(i)  # Flash the corresponding LED
                user_input.append(i)  # Add the index of the pressed button

                # Check immediately after each input
                if user_input[-1] != sequence[len(user_input) - 1]:
                    return False  # Mistake detected
                time.sleep(0.2)  # Debounce delay
                while button.value() == 1:  # Wait for button release
                    time.sleep(0.1)
    return True  # Input complete

def check_sequence():
    return user_input == sequence

def idle_mode():
    while True:
        for i in range(len(leds)):
            leds[i].on()
            time.sleep(0.1)  # Shorter delay for idle mode
            leds[i].off()
            time.sleep(0.1)
        # Check for button press to start the game
        if any(button.value() == 1 for button in buttons):
            break

def game_over():
    # Flash all LEDs together for game over
    for _ in range(3):  # Flash 3 times
        for led in leds:
            led.on()
        play_tone(300)  # Play a different tone for game over
        time.sleep(0.5)
        for led in leds:
            led.off()
        time.sleep(0.5)
```

Simon Says Hardware

```python

def play_game():
    global sequence  # Ensure we modify the global sequence

    # Reset the sequence at the start of each game
    sequence = []

    while True:
        # Generate a new LED index that is not already in the sequence
        next_led = random.randint(0, 3)  # Random index for LEDs
        sequence.append(next_led)
        print("Sequence:", sequence)  # Debugging line
        display("You are in","Level "+str(len(sequence)))

        # Display the sequence
        for led_index in sequence:
            light_led(led_index)
            time.sleep(0.01)

        # Get user input
        if not get_user_input():  # Check for mistakes immediately
            print("Game Over! You failed.")
            display("Game Over!",str(len(sequence)-1)+ " Points")
            game_over()  # Flash LEDs for game over
            break  # Exit game to return to idle mode

        print("Correct! Next round.")

```

```python
# Main loop
while True:
    display("Simon says","press a button")
    idle_mode()  # Start in idle mode
    time.sleep(1)  # Wait for 1 second before starting the game
    play_game()  # Start the game when a button is pressed
```

Challenge II

- Spiele Lieblingslied auf passivem Buzzer
    - Nutze einen KI-Agenten wie die Microsoft CoPilot Webseite um ein MicroPython Programm zu generieren
    - Passe Pin an Platine an

Challenge II

Erstelle ein MicroPython Programm für den Raspberry Pico das das Lied "Happy Birthday" auf einem passiven Buzzer an GP14 abspielt

Gern! Hier ist ein vollständiges, funktionsfähiges **MicroPython-Programm für den Raspberry Pi Pico**, das **„Happy Birthday“** *instrumental* (also ohne Liedtext) über einen **passiven Buzzer an GP14** abspielt.

Der Code nutzt PWM, erzeugt die benötigten Frequenzen und spielt die Melodie in der üblichen Tonfolge.

🎵 MicroPython-Programm: „Happy Birthday“ auf passivem Buzzer (GP14)

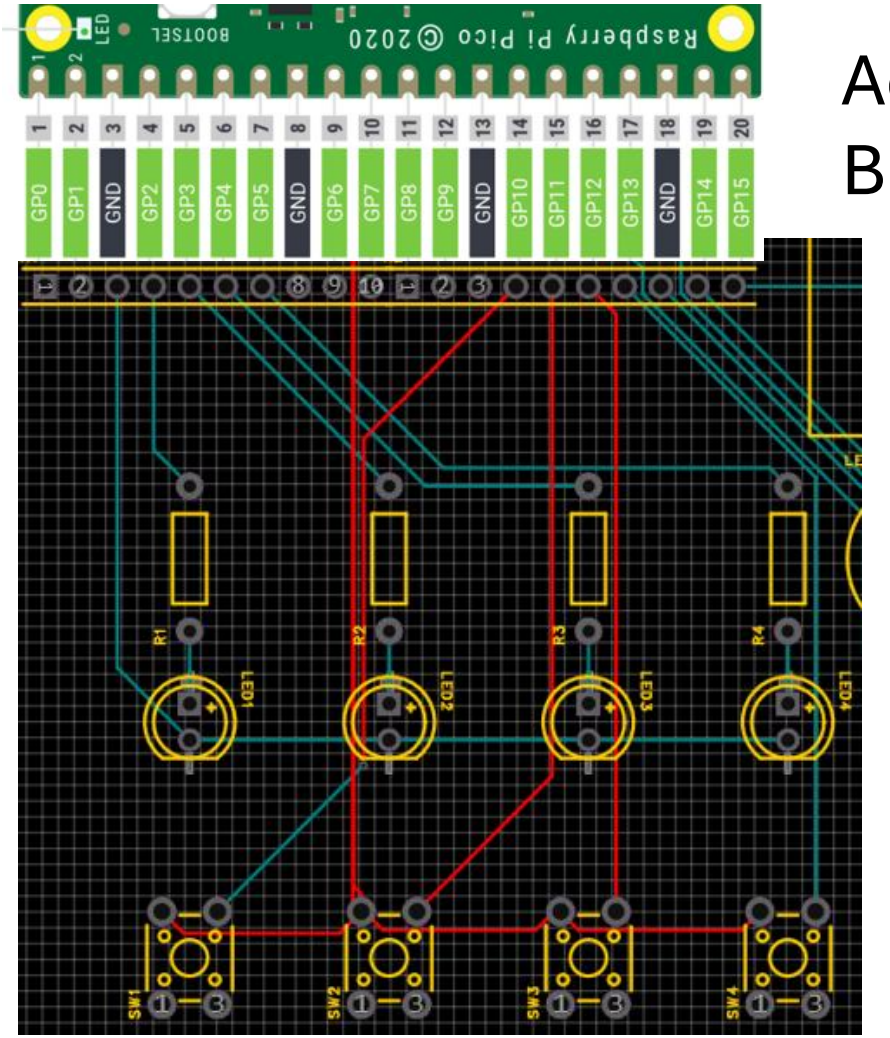
Python Kopieren

```python
from machine import Pin, PWM
from time import sleep
```

Challenge III

- Portiere Reaction Game auf Platine
  - Entscheide welche Buttons verwendet werden sollen
  - Passen Pins an

Challenge III



Achtung:
Buttons sind mit 3V3 statt mit GND verbunden

Challenge IV

- Erreiche 10 Punkte im Simon Says Spiel

Challenge IV